Portfolio Inclusion Report

Kai Chi
CSE 535: Mobile Computing
Arizona State University
Tempe, Arizona
kchi5@asu.edu

Abstract – We create an android app that takes the video of a hand gesture and uploads it to a server. Then we use tensorflow to do image recognition on the video and identify the gesture.

Keywords – android, app, image recognition

I. INTRODUCTION

A. Part 1: The App

For Part 1 of this project, we are asked to develop a mobile application with the listed functionalities: [1] A. The user is shown a video of a gesture; B. The user can replay the video at least three (3) times; C. Upon clicking the "PRACTICE" button, the user can capture his or her own video through the smartphone's front camera for a period of at most five (5) seconds; D. The videos are uploaded to a server.

The mobile application should have three (3) screens. Screen 1: A drop-down menu of 17 different gestures will be shown on this screen. Once a single gesture is selected, the user will be taken to Screen 2. Screen 2: The video of an expert performing the gesture will be shown on this screen. Screen 2 will have another button that says "PRACTICE". Once this button is pressed, the user will be taken to Screen 3. Screen 3: In this screen, the camera interface will be opened for the user to record the practice gesture. The video will be captured for five (5) seconds, and the video will be saved with this filename format:

[GESTURE NAME]_PRACTICE_[practice number]_[USER LASTNAME].mp4

Screen 3 will have another button that says "UPLOAD". Once this button is pressed, the user will be able to upload the gesture to a local server. Moreover, clicking this button will take the user back to Screen 1.

B. Part 2: Image Recognition

For part 2 of this project, we're asked to develop a Python application for classifying SmartHome gestures, using the following steps: [2] 1) Extract the middle frames of all the training gesture videos. 2) For each gesture video, you will have one frame extract the hand shape feature by calling the get_Intsance() method of the HandShapeFeatureExtractor class. (HandShapeFeatureExtractor class uses CNN model that is trained for alphabet gestures). 3) For each gesture, extract the feature vector. 4) Feature vectors of all the gestures is the penultimate layer of the training set. 5) Apply cosine similarity between the vector of the gesture video and the penultimate layer of the training set. Corresponding gesture of the training set vector with minimum cosine difference is the recognition of the gesture. 6) Save the gesture number to the "Results.csv" 7) Recognize the gestures for all the test dataset videos and save the results to the "Results.csv" file.

II. EXPLANATION OF THE SOLUTION

A. Part 1: The App

We create an android app from scratch. It shows the user a video of a gesture and then records the user doing the gesture. See "MainActivity.java" for details.

On screen 1, the user enters the ip address of a server and their last name. The app uses the java.net.HttpURLConnection [3] module to send data to the server at the given ip address.



Fig. 1: Screen 1

On screen 2, the user chooses from a drop-down menu of 17 different gestures. Once a single gesture is selected, the user can play a video of the gesture. The user can replay the video at least three times.



Fig. 2: Screen 2

On screen 3, the user can capture their own video through the smartphone's front camera for a period of at most five seconds. The app uses the android.hardware.camera2.CameraDevice [4] module to open the camera, and uses the android.media.MediaRecorder [5] module to record the video. The app uses the android.os.CountDownTimer [6] module to stop recording after five seconds.



Fig.3: Screen 3

On screen 4, the user can replay the video and upload it to a server. We decide not to take the user back to screen 1 when clicking the "upload" because we want to display an error message in case the upload fails.



Fig. 4: Screen 4

The file names are automatically generated based on the gesture name and the user's last name and take number.

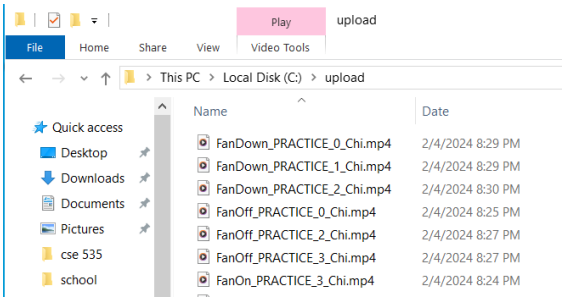


Fig. 5: File Names

The server uses a simple php script to catch the uploaded file and save it to a folder. See “upload.php” for details.

One problem we have is the app installation size is as large as 122MB. This happens because the gesture videos are shipped with the app. The video files are stored in the asset folder of the app. In future iterations, we can move the videos online and access them via URL to reduce the installation size.

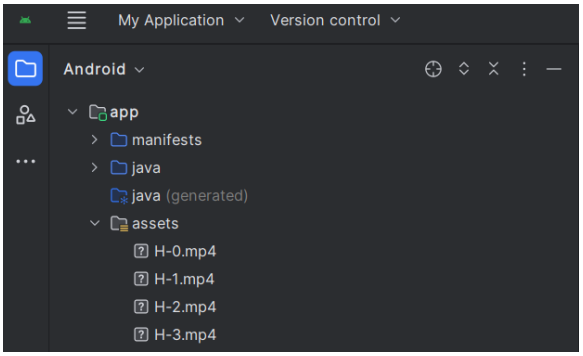


Fig. 6: App Assets

B. Part 2: Image Recognition

For part 2, we’re asked to create a machine learning model to detect the gesture in a given video. We have 68 training videos and there are 17 different gestures.

We start by coding up a feature extractor. See “frameextractor.py” for details. It takes the middle frame of the video and resizes it to 200x200 in RGB. We get the class label from the file name of the video.



Fig. 7: Middle Frame

We use tensorflow to create a neural network. See “main.py” for details. It takes an input of size 200x200x3. It does 2 rounds of convolution and 2 rounds of maxpooling. Then it flattens the array and feeds forward to a hidden, fully connected layer of 1000 neurons. The output layer size is 17x1. It uses the softmax function to produce the probabilities. The predicted class is the one with the highest probability. [7][9]

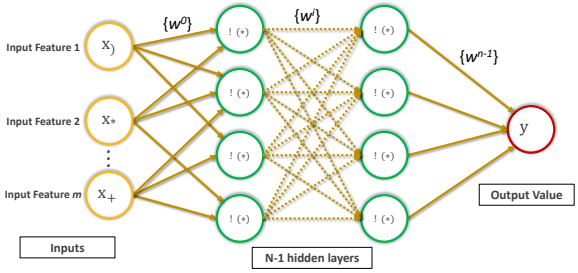


Fig. 8: Neural Network

We specify a loss function in compile() and use the built-in train() function to train the model. The neural network learns the weights by using some form of gradient descent to minimize some loss function. [7][8] After 50 epochs, it has an accuracy of about 100% on the training set.

We take this model and use it to do prediction on the test set. Unfortunately, the accuracy is only about 64%. If we have more

training videos, we may be able to achieve a higher test accuracy score.

III. DESCRIPTION OF THE RESULTS

Part 1

The app and server both work very well. The app is able to take a video and upload to the server. The server is able to save the video to a folder on the computer. File name is incremented automatically.

Part 2

The feature extraction part works well. The image recognition part is more difficult. Our accuracy is about 64%. This is because we only have 68 training videos for 17 classes, or 4 videos per class. Ideally, we want 100 samples per class. The accuracy can be improved if we have more training videos.

IV. DESCRIPTION OF YOUR CONTRIBUTIONS TO THE PROJECT

This is not a group project. I am the sole contributor to the project.

V. EXPLANATION OF WHAT NEW SKILLS, TECHNIQUES, OR KNOWLEDGE YOU ACQUIRED FROM THE PROJECT

This project allows us to practice writing an android app in Java and doing server work using php. It also allows us to practice using tensorflow to do machine learning. Finally, it trains us to use github for version control.

REFERENCES

[1] CSE 535_SmartHome Gesture Control Application Project Part 1_Overview Document.pdf

https://asu.instructure.com/courses/176052/files/80113648?wrap=1

[2] CSE 535_SmartHome Gesture Control Application Project Part 2_Overview Document.pdf

https://asu.instructure.com/courses/176052/files/80113651?wrap=1

[3] java.net.HttpURLConnection

https://developer.android.com/reference/java/net/HttpURLConnection

[4][5] android.hardware.camera2, android.media.MediaRecorder

https://github.com/android/camera-samples/tree/main/Camera2Video

[6] android.os.CountDownTimer

https://developer.android.com/reference/android/os/CountDownTimer

[7] Built-in methods

https://www.tensorflow.org/guide/keras/training_with_built_in_methods

[8] Adam optimization

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam

[9] Bishop, Christopher M. Pattern Recognition and Machine Learning. Springer, 2006.